```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import pandas as pd
import matplotlib.pyplot as plt

#debugging
plt.ion()

progress =[]


class ImageDataset(Dataset):
    def __init__(self, csv_file):
        self.data = pd.read_csv(csv_file)

        self.features = self.data.iloc[:, :-1].values.astype('float32')
        self.labels = self.data.iloc[:, -1].values.astype('int64')

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        x = self.features[idx]
        y = self.labels[idx]
        return torch.tensor(x), torch.tensor(y)

class MLPClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(MLPClassifier, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size)
        )

    def forward(self, x):
        return self.model(x)

def plot_data(x,y):
    progress.append([x,y])
    plt.plot([i[0] for i in progress],[i[1] for i in progress])
    plt.show()

def train_model(model, dataloader, criterion, optimizer, epochs):
    losses = []
    fig, ax = plt.subplots()

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for inputs, labels in dataloader:
            inputs, labels = inputs.to(device), labels.to(device)

            outputs = model(inputs)
            loss = criterion(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        epoch_loss = total_loss / len(dataloader)
        losses.append(epoch_loss)

        ax.clear()
        ax.plot(range(1, len(losses) + 1), losses)
        ax.set_xlabel('Epoch')
        ax.set_ylabel('Loss')
        ax.set_title('Training Loss vs. Epoch')
        plt.draw()
        plt.pause(0.001)

        print(f"Epoch [{epoch + 1}/{epochs}], Loss: {epoch_loss:.4f}")

    plt.ioff()

def evaluate_model(model, dataloader):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for inputs, labels in dataloader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    print(f"Accuracy: {accuracy:.2f}%")

if __name__ == "__main__":
    csv_file = "pixel_data_512x384.csv"
    input_size = 36864
    hidden_size = 64
    num_classes = 12
    batch_size = 32
    learning_rate = 0.0005
    epochs = 400

    # Check for GPU
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    if device.type == "cuda":
        print(f"Using GPU: {torch.cuda.get_device_name(0)}")
    else:
        print("Using CPU")

    dataset = ImageDataset(csv_file)
```

```
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

model = MLPClassifier(input_size, hidden_size, num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

train_model(model, dataloader, criterion, optimizer, epochs)

evaluate_model(model, dataloader)
```

In this notebook, we develop a Convolutional Neural Network (CNN) to classify images of waste into six categories: glass, metal, paper, plastic, cardboard, and general trash.

We first need to import the neccessary libraries to create the convolutional neural network using pytorch.

```
import os
import cv2
import numpy as np
from tqdm import tqdm
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset, random_split
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
from sklearn.metrics import confusion_matrix
import seaborn as sn
import matplotlib.pyplot as plt
import random
import pandas as pd

import torch
print("CUDA available:", torch.cuda.is_available())
print("CUDA device name:", torch.cuda.get_device_name(0) if
torch.cuda.is_available() else "Not detected")
```

```
CUDA available: True
CUDA device name: NVIDIA GeForce RTX 4060 Ti
```

Now that we have the relevant imports, we can create a garbage dataset class that contains information such as labels and images. In addition, the class should contain a method for returning the length and for fetching information about a specific item.

```
def load_and_preprocess_data(data_dir, img_size=(64, 64)):
    X = []
    y = []
    categories = []

    for d in os.listdir(data_dir):
        if os.path.isdir(os.path.join(data_dir, d)) and not
d.startswith('.'):
            categories.append(d)

    print("Found categories:", categories)
    categories.sort()

    print("Loading and preprocessing images...")
    for category_idx, category in enumerate(categories):
```

```
        category_path = os.path.join(data_dir, category)
        if not os.path.isdir(category_path):
            continue

        print(f"Processing Category: {category} ({category_idx})")
        image_files = [f for f in os.listdir(category_path) if
f.lower().endswith('.jpg')]

        for img_file in tqdm(image_files[:500]):
            img_path = os.path.join(category_path, img_file)
            img = cv2.imread(img_path)
            img = cv2.resize(img, img_size)
            img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            img_rgb = img_rgb / 255.0

            X.append(img_rgb)
            y.append(category_idx)

    return np.array(X), np.array(y), categories
```

This function loads and preprocesses image data from a specified directory. In this case each subdirectory represents a category. It reads up to 500 ".jpg" images per category, resizing them to 512x512 pixels converting them to RGB and normalizes pixel values (between 0 and 1). The dimensions of the input can be modified.

Now we define the CNN Model.

```
class CNN(nn.Module):
    def __init__(self, num_classes=6):
        super(CNN, self).__init__()

        self.conv_layers = nn.Sequential(
            # Block 1: Input (3, 64, 64) Output (32 , 32, 32)
            nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3,
padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2), # Reduces to 32 x32
            nn.Dropout2d(0.1),

            # Block 2: Input (32, 32, 32) Output (64, 16, 16)
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2), # Reduces to 16x16
            nn.Dropout2d(0.1),

            # Block 3: Input (64, 16, 16) Output (128, 8 , 8)
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
```

```
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2), # Reduces to 8x8
            nn.Dropout2d(0.2),

            # Block 4: Input (128, 8, 8)  Output (256, 4, 4)
            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.MaxPool2d(2), # Reduces to 4x4
            nn.Dropout2d(0.3),
        )

        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(256 * 4 * 4, 512)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.conv_layers(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

This function evaluates the model on a dataset and returns prediction results. If "verbose" is "true", it prints the accuracy and a detailed classification report.

```
DATA_DIR = "./Garbage classification"
X, y, categories = load_and_preprocess_data(DATA_DIR)
print("Data shape:", X.shape, y.shape)
num_classes = len(categories)

for class_idx in range(len(categories)):
    img = X[y == class_idx][random.randint(0,len(X[y == class_idx])-
1)]
    plt.imshow(img)
    plt.title(f"Class: {categories[class_idx]}")
    plt.axis('off')
    plt.show()
```

```
Found categories: ['cardboard', 'glass', 'metal', 'paper', 'plastic',
'trash']
Loading and preprocessing images...
Processing Category: cardboard (0)
```

```
100%|██████████| 403/403 [00:00<00:00, 477.45it/s]
100%|██████████| 403/403 [00:00<00:00, 477.45it/s]

Processing Category: glass (1)

100%|██████████| 500/500 [00:01<00:00, 476.07it/s]
100%|██████████| 500/500 [00:01<00:00, 476.07it/s]

Processing Category: metal (2)

100%|██████████| 410/410 [00:00<00:00, 460.47it/s]
100%|██████████| 410/410 [00:00<00:00, 460.47it/s]

Processing Category: paper (3)

100%|██████████| 500/500 [00:01<00:00, 402.72it/s]


Processing Category: plastic (4)

100%|██████████| 482/482 [00:01<00:00, 389.26it/s]
100%|██████████| 482/482 [00:01<00:00, 389.26it/s]

Processing Category: trash (5)

100%|██████████| 137/137 [00:00<00:00, 398.47it/s]


Data shape: (2432, 512, 512, 3) (2432,)
```

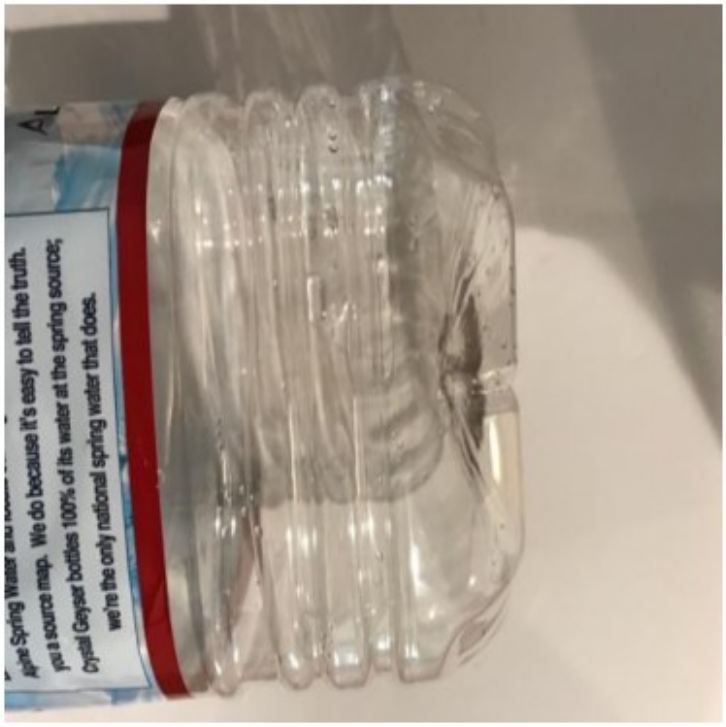
Class: cardboard

Class: glass

Class: metal

Class: paper

Class: plastic



Class: trash

```python
# Train/test split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
stratify=y, random_state=42)
X_train_tensor = torch.tensor(X_train.transpose(0, 3, 1, 2),
dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.long)
X_val_tensor = torch.tensor(X_val.transpose(0, 3, 1, 2),
dtype=torch.float32)
y_val_tensor = torch.tensor(y_val, dtype=torch.long)

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = CNN(num_classes=num_classes).to(device)

train_model(model, train_loader, val_loader, num_epochs=20)

torch.save(model.state_dict(), "cnn_model.pth")
```

```
Epoch [1/20], Loss: 1.6639
Accuracy: 0.4702258726899384
Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.67      0.72        81
           1       0.36      0.58      0.45       100
           2       0.64      0.09      0.15        82
           3       0.69      0.38      0.49       100
           4       0.38      0.74      0.50        97
           5       0.00      0.00      0.00        27

    accuracy                           0.47       487
   macro avg       0.47      0.41      0.38       487
weighted avg       0.53      0.47      0.44       487

Accuracy: 0.4702258726899384
Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.67      0.72        81
           1       0.36      0.58      0.45       100
           2       0.64      0.09      0.15        82
           3       0.69      0.38      0.49       100
           4       0.38      0.74      0.50        97
           5       0.00      0.00      0.00        27

    accuracy                           0.47       487
```

```
   macro avg       0.47      0.41      0.38       487
weighted avg       0.53      0.47      0.44       487

Epoch [2/20], Loss: 1.4612
Epoch [2/20], Loss: 1.4612
Accuracy: 0.5359342915811088
Classification Report:
              precision    recall  f1-score   support

           0       0.71      0.79      0.75        81
           1       0.42      0.65      0.51       100
           2       0.63      0.21      0.31        82
           3       0.73      0.51      0.60       100
           4       0.45      0.66      0.53        97
           5       0.00      0.00      0.00        27

    accuracy                           0.54       487
   macro avg       0.49      0.47      0.45       487
weighted avg       0.55      0.54      0.51       487

Accuracy: 0.5359342915811088
Classification Report:
              precision    recall  f1-score   support

           0       0.71      0.79      0.75        81
           1       0.42      0.65      0.51       100
           2       0.63      0.21      0.31        82
           3       0.73      0.51      0.60       100
           4       0.45      0.66      0.53        97
           5       0.00      0.00      0.00        27

    accuracy                           0.54       487
   macro avg       0.49      0.47      0.45       487
weighted avg       0.55      0.54      0.51       487

Epoch [3/20], Loss: 1.3623
Epoch [3/20], Loss: 1.3623
Accuracy: 0.5462012320328542
Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.74      0.76        81
           1       0.46      0.63      0.53       100
           2       0.62      0.22      0.32        82
           3       0.74      0.57      0.64       100
           4       0.41      0.70      0.52        97
           5       0.00      0.00      0.00        27

    accuracy                           0.55       487
   macro avg       0.50      0.48      0.46       487
```

```
weighted avg       0.56      0.55      0.53       487

Accuracy: 0.5462012320328542
Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.74      0.76        81
           1       0.46      0.63      0.53       100
           2       0.62      0.22      0.32        82
           3       0.74      0.57      0.64       100
           4       0.41      0.70      0.52        97
           5       0.00      0.00      0.00        27

    accuracy                           0.55       487
   macro avg       0.50      0.48      0.46       487
weighted avg       0.56      0.55      0.53       487

Epoch [4/20], Loss: 1.2966
Epoch [4/20], Loss: 1.2966
Accuracy: 0.5790554414784395
Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.75      0.77        81
           1       0.40      0.70      0.51       100
           2       0.68      0.28      0.40        82
           3       0.80      0.66      0.72       100
           4       0.52      0.64      0.57        97
           5       0.00      0.00      0.00        27

    accuracy                           0.58       487
   macro avg       0.53      0.51      0.50       487
weighted avg       0.60      0.58      0.56       487

Accuracy: 0.5790554414784395
Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.75      0.77        81
           1       0.40      0.70      0.51       100
           2       0.68      0.28      0.40        82
           3       0.80      0.66      0.72       100
           4       0.52      0.64      0.57        97
           5       0.00      0.00      0.00        27

    accuracy                           0.58       487
   macro avg       0.53      0.51      0.50       487
weighted avg       0.60      0.58      0.56       487

Epoch [5/20], Loss: 1.2632
```

```
Epoch [5/20], Loss: 1.2632
Accuracy: 0.6324435318275154
Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.74      0.76        81
           1       0.45      0.75      0.56       100
           2       0.68      0.30      0.42        82
           3       0.84      0.87      0.85       100
           4       0.60      0.63      0.61        97
           5       0.00      0.00      0.00        27

    accuracy                           0.63       487
   macro avg       0.56      0.55      0.54       487
weighted avg       0.63      0.63      0.61       487

Accuracy: 0.6324435318275154
Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.74      0.76        81
           1       0.45      0.75      0.56       100
           2       0.68      0.30      0.42        82
           3       0.84      0.87      0.85       100
           4       0.60      0.63      0.61        97
           5       0.00      0.00      0.00        27

    accuracy                           0.63       487
   macro avg       0.56      0.55      0.54       487
weighted avg       0.63      0.63      0.61       487

Epoch [6/20], Loss: 1.2218
Epoch [6/20], Loss: 1.2218
Accuracy: 0.6509240246406571
Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.77      0.79        81
           1       0.53      0.69      0.60       100
           2       0.68      0.39      0.50        82
           3       0.73      0.88      0.80       100
           4       0.59      0.67      0.63        97
           5       0.50      0.04      0.07        27

    accuracy                           0.65       487
   macro avg       0.64      0.57      0.56       487
weighted avg       0.65      0.65      0.63       487

Accuracy: 0.6509240246406571
Classification Report:
```

```
              precision    recall  f1-score   support

           0       0.82      0.77      0.79        81
           1       0.53      0.69      0.60       100
           2       0.68      0.39      0.50        82
           3       0.73      0.88      0.80       100
           4       0.59      0.67      0.63        97
           5       0.50      0.04      0.07        27

    accuracy                           0.65       487
   macro avg       0.64      0.57      0.56       487
weighted avg       0.65      0.65      0.63       487

Epoch [7/20], Loss: 1.1963
Epoch [7/20], Loss: 1.1963
Accuracy: 0.5913757700205339
Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.77      0.81        81
           1       0.46      0.76      0.57       100
           2       0.49      0.49      0.49        82
           3       0.92      0.46      0.61       100
           4       0.56      0.66      0.60        97
           5       0.00      0.00      0.00        27

    accuracy                           0.59       487
   macro avg       0.55      0.52      0.51       487
weighted avg       0.62      0.59      0.58       487

Accuracy: 0.5913757700205339
Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.77      0.81        81
           1       0.46      0.76      0.57       100
           2       0.49      0.49      0.49        82
           3       0.92      0.46      0.61       100
           4       0.56      0.66      0.60        97
           5       0.00      0.00      0.00        27

    accuracy                           0.59       487
   macro avg       0.55      0.52      0.51       487
weighted avg       0.62      0.59      0.58       487

Epoch [8/20], Loss: 1.1624
Epoch [8/20], Loss: 1.1624
Accuracy: 0.6570841889117043
Classification Report:
              precision    recall  f1-score   support
```

```
              0       0.84      0.78      0.81        81
              1       0.51      0.70      0.59       100
              2       0.59      0.44      0.50        82
              3       0.79      0.84      0.82       100
              4       0.61      0.65      0.63        97
              5       0.80      0.15      0.25        27

    accuracy                           0.66       487
   macro avg       0.69      0.59      0.60       487
weighted avg       0.67      0.66      0.65       487

Accuracy: 0.6570841889117043
Classification Report:
              precision    recall  f1-score   support

              0       0.84      0.78      0.81        81
              1       0.51      0.70      0.59       100
              2       0.59      0.44      0.50        82
              3       0.79      0.84      0.82       100
              4       0.61      0.65      0.63        97
              5       0.80      0.15      0.25        27

    accuracy                           0.66       487
   macro avg       0.69      0.59      0.60       487
weighted avg       0.67      0.66      0.65       487

Epoch [9/20], Loss: 1.1359
Epoch [9/20], Loss: 1.1359
Accuracy: 0.6591375770020534
Classification Report:
              precision    recall  f1-score   support

              0       0.85      0.78      0.81        81
              1       0.49      0.77      0.60       100
              2       0.62      0.41      0.50        82
              3       0.82      0.85      0.83       100
              4       0.63      0.62      0.62        97
              5       0.67      0.07      0.13        27

    accuracy                           0.66       487
   macro avg       0.68      0.58      0.58       487
weighted avg       0.68      0.66      0.65       487

Accuracy: 0.6591375770020534
Classification Report:
              precision    recall  f1-score   support

              0       0.85      0.78      0.81        81
              1       0.49      0.77      0.60       100
```

```
           2       0.62      0.41      0.50        82
           3       0.82      0.85      0.83       100
           4       0.63      0.62      0.62        97
           5       0.67      0.07      0.13        27

    accuracy                           0.66       487
   macro avg       0.68      0.58      0.58       487
weighted avg       0.68      0.66      0.65       487

Epoch [10/20], Loss: 1.1338
Epoch [10/20], Loss: 1.1338
Accuracy: 0.6714579055441479
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.79      0.83        81
           1       0.54      0.75      0.63       100
           2       0.64      0.39      0.48        82
           3       0.74      0.90      0.81       100
           4       0.63      0.64      0.63        97
           5       0.80      0.15      0.25        27

    accuracy                           0.67       487
   macro avg       0.70      0.60      0.61       487
weighted avg       0.69      0.67      0.66       487

Accuracy: 0.6714579055441479
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.79      0.83        81
           1       0.54      0.75      0.63       100
           2       0.64      0.39      0.48        82
           3       0.74      0.90      0.81       100
           4       0.63      0.64      0.63        97
           5       0.80      0.15      0.25        27

    accuracy                           0.67       487
   macro avg       0.70      0.60      0.61       487
weighted avg       0.69      0.67      0.66       487

Epoch [11/20], Loss: 1.0999
Epoch [11/20], Loss: 1.0999
Accuracy: 0.6652977412731006
Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.74      0.79        81
           1       0.59      0.69      0.64       100
           2       0.64      0.39      0.48        82
```

```
           3       0.64      0.95      0.76       100
           4       0.66      0.66      0.66        97
           5       1.00      0.15      0.26        27

    accuracy                           0.67       487
   macro avg       0.73      0.60      0.60       487
weighted avg       0.69      0.67      0.65       487

Accuracy: 0.6652977412731006
Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.74      0.79        81
           1       0.59      0.69      0.64       100
           2       0.64      0.39      0.48        82
           3       0.64      0.95      0.76       100
           4       0.66      0.66      0.66        97
           5       1.00      0.15      0.26        27

    accuracy                           0.67       487
   macro avg       0.73      0.60      0.60       487
weighted avg       0.69      0.67      0.65       487

Epoch [12/20], Loss: 1.0743
Epoch [12/20], Loss: 1.0743
Accuracy: 0.6776180698151951
Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.80      0.82        81
           1       0.56      0.75      0.64       100
           2       0.60      0.43      0.50        82
           3       0.74      0.90      0.81       100
           4       0.67      0.63      0.65        97
           5       1.00      0.15      0.26        27

    accuracy                           0.68       487
   macro avg       0.74      0.61      0.61       487
weighted avg       0.70      0.68      0.66       487

Accuracy: 0.6776180698151951
Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.80      0.82        81
           1       0.56      0.75      0.64       100
           2       0.60      0.43      0.50        82
           3       0.74      0.90      0.81       100
           4       0.67      0.63      0.65        97
           5       1.00      0.15      0.26        27
```

```
    accuracy                           0.68       487
   macro avg       0.74      0.61      0.61       487
weighted avg       0.70      0.68      0.66       487

Epoch [13/20], Loss: 1.0707
Epoch [13/20], Loss: 1.0707
Accuracy: 0.6714579055441479
Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.84      0.82        81
           1       0.57      0.69      0.63       100
           2       0.55      0.39      0.46        82
           3       0.75      0.86      0.80       100
           4       0.63      0.69      0.66        97
           5       1.00      0.19      0.31        27

    accuracy                           0.67       487
   macro avg       0.72      0.61      0.61       487
weighted avg       0.68      0.67      0.66       487

Accuracy: 0.6714579055441479
Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.84      0.82        81
           1       0.57      0.69      0.63       100
           2       0.55      0.39      0.46        82
           3       0.75      0.86      0.80       100
           4       0.63      0.69      0.66        97
           5       1.00      0.19      0.31        27

    accuracy                           0.67       487
   macro avg       0.72      0.61      0.61       487
weighted avg       0.68      0.67      0.66       487

Epoch [14/20], Loss: 1.0523
Epoch [14/20], Loss: 1.0523
Accuracy: 0.6735112936344969
Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.83      0.83        81
           1       0.53      0.80      0.63       100
           2       0.54      0.48      0.51        82
           3       0.93      0.75      0.83       100
           4       0.65      0.66      0.65        97
           5       1.00      0.11      0.20        27
```

```
    accuracy                           0.67       487
   macro avg       0.75      0.60      0.61       487
weighted avg       0.71      0.67      0.67       487

Accuracy: 0.6735112936344969
Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.83      0.83        81
           1       0.53      0.80      0.63       100
           2       0.54      0.48      0.51        82
           3       0.93      0.75      0.83       100
           4       0.65      0.66      0.65        97
           5       1.00      0.11      0.20        27

    accuracy                           0.67       487
   macro avg       0.75      0.60      0.61       487
weighted avg       0.71      0.67      0.67       487

Epoch [15/20], Loss: 1.0527
Epoch [15/20], Loss: 1.0527
Accuracy: 0.6837782340862423
Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.78      0.84        81
           1       0.65      0.62      0.64       100
           2       0.64      0.46      0.54        82
           3       0.62      0.96      0.76       100
           4       0.66      0.69      0.67        97
           5       0.88      0.26      0.40        27

    accuracy                           0.68       487
   macro avg       0.73      0.63      0.64       487
weighted avg       0.70      0.68      0.67       487

Accuracy: 0.6837782340862423
Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.78      0.84        81
           1       0.65      0.62      0.64       100
           2       0.64      0.46      0.54        82
           3       0.62      0.96      0.76       100
           4       0.66      0.69      0.67        97
           5       0.88      0.26      0.40        27

    accuracy                           0.68       487
   macro avg       0.73      0.63      0.64       487
weighted avg       0.70      0.68      0.67       487
```

```
Epoch [16/20], Loss: 1.0180
Epoch [16/20], Loss: 1.0180
Accuracy: 0.6919917864476386
Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.83      0.85        81
           1       0.66      0.67      0.67       100
           2       0.67      0.40      0.50        82
           3       0.63      0.96      0.76       100
           4       0.68      0.67      0.67        97
           5       0.82      0.33      0.47        27

    accuracy                           0.69       487
   macro avg       0.72      0.64      0.65       487
weighted avg       0.70      0.69      0.68       487

Accuracy: 0.6919917864476386
Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.83      0.85        81
           1       0.66      0.67      0.67       100
           2       0.67      0.40      0.50        82
           3       0.63      0.96      0.76       100
           4       0.68      0.67      0.67        97
           5       0.82      0.33      0.47        27

    accuracy                           0.69       487
   macro avg       0.72      0.64      0.65       487
weighted avg       0.70      0.69      0.68       487

Epoch [17/20], Loss: 1.0007
Epoch [17/20], Loss: 1.0007
Accuracy: 0.7125256673511293
Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.78      0.85        81
           1       0.59      0.83      0.69       100
           2       0.67      0.49      0.56        82
           3       0.70      0.94      0.80       100
           4       0.77      0.60      0.67        97
           5       1.00      0.33      0.50        27

    accuracy                           0.71       487
   macro avg       0.78      0.66      0.68       487
weighted avg       0.74      0.71      0.70       487

Accuracy: 0.7125256673511293
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.78      0.85        81
           1       0.59      0.83      0.69       100
           2       0.67      0.49      0.56        82
           3       0.70      0.94      0.80       100
           4       0.77      0.60      0.67        97
           5       1.00      0.33      0.50        27

    accuracy                           0.71       487
   macro avg       0.78      0.66      0.68       487
weighted avg       0.74      0.71      0.70       487

Epoch [18/20], Loss: 0.9872
Epoch [18/20], Loss: 0.9872
Accuracy: 0.6817248459958932
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.83      0.85        81
           1       0.52      0.80      0.63       100
           2       0.53      0.51      0.52        82
           3       0.94      0.75      0.83       100
           4       0.69      0.67      0.68        97
           5       1.00      0.11      0.20        27

    accuracy                           0.68       487
   macro avg       0.76      0.61      0.62       487
weighted avg       0.73      0.68      0.68       487

Accuracy: 0.6817248459958932
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.83      0.85        81
           1       0.52      0.80      0.63       100
           2       0.53      0.51      0.52        82
           3       0.94      0.75      0.83       100
           4       0.69      0.67      0.68        97
           5       1.00      0.11      0.20        27

    accuracy                           0.68       487
   macro avg       0.76      0.61      0.62       487
weighted avg       0.73      0.68      0.68       487

Epoch [19/20], Loss: 0.9903
Epoch [19/20], Loss: 0.9903
Accuracy: 0.702258726899384
Classification Report:
```

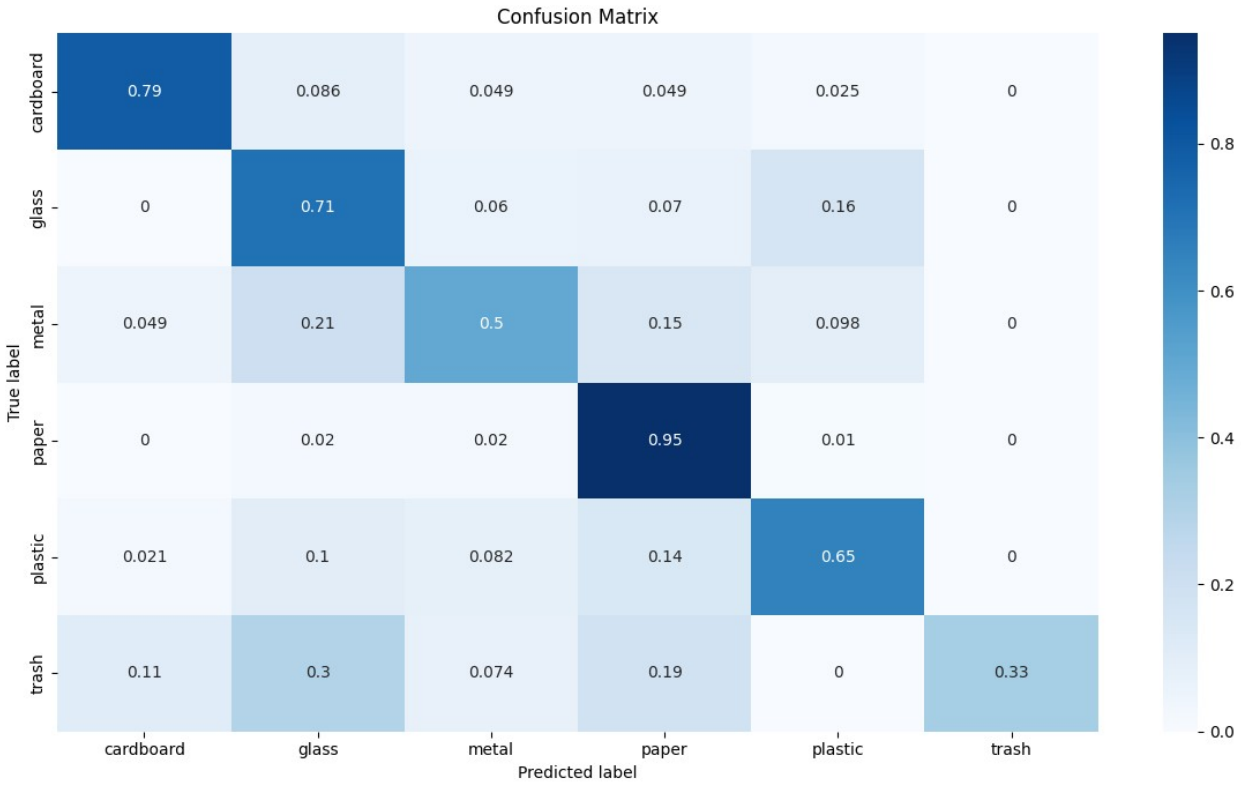
```
              precision    recall  f1-score   support

           0       0.88      0.84      0.86        81
           1       0.58      0.76      0.66       100
           2       0.60      0.52      0.56        82
           3       0.84      0.81      0.82       100
           4       0.64      0.69      0.67        97
           5       1.00      0.26      0.41        27

    accuracy                           0.70       487
   macro avg       0.76      0.65      0.66       487
weighted avg       0.72      0.70      0.70       487

Accuracy: 0.702258726899384
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.84      0.86        81
           1       0.58      0.76      0.66       100
           2       0.60      0.52      0.56        82
           3       0.84      0.81      0.82       100
           4       0.64      0.69      0.67        97
           5       1.00      0.26      0.41        27

    accuracy                           0.70       487
   macro avg       0.76      0.65      0.66       487
weighted avg       0.72      0.70      0.70       487

Epoch [20/20], Loss: 0.9726
Epoch [20/20], Loss: 0.9726
Accuracy: 0.704312114989733
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.79      0.83        81
           1       0.62      0.71      0.66       100
           2       0.65      0.50      0.57        82
           3       0.69      0.95      0.80       100
           4       0.70      0.65      0.67        97
           5       1.00      0.33      0.50        27

    accuracy                           0.70       487
   macro avg       0.76      0.66      0.67       487
weighted avg       0.72      0.70      0.70       487

Accuracy: 0.704312114989733
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.79      0.83        81
```

```
           1       0.62      0.71      0.66       100
           2       0.65      0.50      0.57        82
           3       0.69      0.95      0.80       100
           4       0.70      0.65      0.67        97
           5       1.00      0.33      0.50        27

    accuracy                           0.70       487
   macro avg       0.76      0.66      0.67       487
weighted avg       0.72      0.70      0.70       487
```

This code loads and preprocesses image data, then splits it into training and validation sets. It converts the data into PyTorch tensors and creates data loaders for batching. The model is initialized on GPU if available, trained on the training set, and evaluated on the validation set. Finally, the trained model is saved to a file.

```
y_pred = []
y_true = []

model.eval()

with torch.no_grad():
    for inputs, labels in val_loader:
        inputs = inputs.to(device)

        outputs = model(inputs)

        _, predicted = torch.max(outputs, 1)
        y_pred.extend(predicted.cpu().numpy())

        labels = labels.numpy()
        y_true.extend(labels)

cf_matrix = confusion_matrix(y_true, y_pred)
df_cm = pd.DataFrame(cf_matrix / np.sum(cf_matrix, axis=1)[:, None],
                     index=[categories[i] for i in
range(len(categories))],
                     columns=[categories[i] for i in
range(len(categories))])

plt.figure(figsize=(12, 7))
sn.heatmap(df_cm, annot=True, cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.tight_layout()
plt.savefig('confusion_matrix.png')
plt.show()
```

```
accuracy = accuracy_score(y_true, y_pred)
print(f"Overall Accuracy: {accuracy:.4f}")
```




```
Overall Accuracy: 0.7043
```

This notebook implements a Multi-Layer Perceptron neural network from scratch for classifying garbage images from the Garbage Classification dataset.

```
import numpy as np
import matplotlib.pyplot as plt
import os
import cv2
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tqdm import tqdm
import seaborn as sns
import random
import zipfile
import time
import sys
import kagglehub

# For reproducibility
np.random.seed(42)
random.seed(42)


DATA_DIR = "./Garbage classification"

print(f"Found data directory: {DATA_DIR}")

print("Directory structure:")
for item in os.listdir(DATA_DIR):
    path = os.path.join(DATA_DIR, item)
    if os.path.isdir(path):
        print(f"  - {item}/ (directory with {len(os.listdir(path))}
items)")
    else:
        print(f"  - {item} (file)")

garbage_dir = os.path.join(DATA_DIR, "Garbage classification")

if os.path.exists(garbage_dir):
    DATA_DIR = garbage_dir
    print(f"Using actual garbage classification directory:
{DATA_DIR}")
    categories = [d for d in os.listdir(DATA_DIR) if
os.path.isdir(os.path.join(DATA_DIR, d))]
    print(f"Found {len(categories)} categories: {categories}")
```

```
Found data directory: ./Garbage classification
Directory structure:
  - cardboard/ (directory with 403 items)
  - glass/ (directory with 501 items)
  - metal/ (directory with 410 items)
```

```
  - paper/ (directory with 594 items)
  - plastic/ (directory with 482 items)
  - trash/ (directory with 137 items)

def load_and_preprocess_data(data_dir, img_size=(128, 128)):
    X = []
    y = []

    categories = [d for d in os.listdir(data_dir)
                  if os.path.isdir(os.path.join(data_dir, d)) and not
d.startswith('.')]
    categories.sort()
    for category_idx, category in enumerate(categories):
        category_path = os.path.join(data_dir, category)

        if not os.path.isdir(category_path):
            continue

        print(f"Processing category: {category} ({category_idx})")
        image_files = [f for f in os.listdir(category_path)
                       if f.lower().endswith(('.jpg'))]
        for img_file in tqdm(image_files[:500]):
            img_path = os.path.join(category_path, img_file)

            img = cv2.imread(img_path)
            if img is None:
                print(f"Warning: Could not read {img_path}")
                continue
            img = cv2.resize(img, img_size)
            flattened_img = img.flatten() / 255.0
            X.append(flattened_img)
            y.append(category_idx)


    return np.array(X), np.array(y), categories

X, y, class_names = load_and_preprocess_data(DATA_DIR)

print(f"Data shape: {X.shape}")
print(f"Labels shape: {y.shape}")
print(f"Number of classes: {len(class_names)}")

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

print(f"Training data: {X_train.shape}")
print(f"Testing data: {X_test.shape}")

Processing category: cardboard (0)
```

```
100%|██████████| 403/403 [00:00<00:00, 1660.67it/s]
100%|██████████| 403/403 [00:00<00:00, 1660.67it/s]

Processing category: glass (1)

100%|██████████| 500/500 [00:00<00:00, 1798.14it/s]
100%|██████████| 500/500 [00:00<00:00, 1798.14it/s]

Processing category: metal (2)

100%|██████████| 410/410 [00:00<00:00, 1730.50it/s]
100%|██████████| 410/410 [00:00<00:00, 1730.50it/s]

Processing category: paper (3)

100%|██████████| 500/500 [00:00<00:00, 1566.98it/s]


Processing category: plastic (4)

100%|██████████| 482/482 [00:00<00:00, 1653.98it/s]
100%|██████████| 482/482 [00:00<00:00, 1653.98it/s]

Processing category: trash (5)

100%|██████████| 137/137 [00:00<00:00, 1556.73it/s]


Data shape: (2432, 49152)
Labels shape: (2432,)
Number of classes: 6
Training data: (1945, 49152)
Testing data: (487, 49152)
```

```python
class MLP:
    def __init__(self, input_size, hidden_sizes, output_size):
        self.layers = []
        self.biases = []
        self.activations = []

        layer_sizes = [input_size] + hidden_sizes + [output_size]

        for i in range(len(layer_sizes) - 1):
            weight = np.random.randn(layer_sizes[i], layer_sizes[i +
1]) * np.sqrt(2. / layer_sizes[i])
            bias = np.zeros((1, layer_sizes[i + 1]))
            self.layers.append(weight)
            self.biases.append(bias)

    def relu(self, x):
        return np.maximum(0, x)
```

```
    def relu_derivative(self, x):
        return (x > 0).astype(float)

    def softmax(self, x):
        exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
        return exp_x / np.sum(exp_x, axis=1, keepdims=True)

    def forward(self, X):
        self.activations = [X]
        a = X
        for i in range(len(self.layers) - 1):
            z = a @ self.layers[i] + self.biases[i]
            a = self.relu(z)
            self.activations.append(a)
        z = a @ self.layers[-1] + self.biases[-1]
        out = self.softmax(z)
        self.activations.append(out)
        return out

    def compute_loss(self, y_pred, y_true):
        m = y_true.shape[0]
        log_probs = -np.log(y_pred[range(m), y_true] + 1e-9)
        return np.sum(log_probs) / m

    def compute_accuracy(self, y_pred, y_true):
        predictions = np.argmax(y_pred, axis=1)
        return np.mean(predictions == y_true)

    def backward(self, y_pred, y_true):
        grads_w = []
        grads_b = []

        m = y_true.shape[0]
        y_true_one_hot = np.zeros_like(y_pred)
        y_true_one_hot[np.arange(m), y_true] = 1

        delta = (y_pred - y_true_one_hot) / m

        for i in reversed(range(len(self.layers))):
            a_prev = self.activations[i]
            dw = a_prev.T @ delta
            db = np.sum(delta, axis=0, keepdims=True)
            grads_w.insert(0, dw)
            grads_b.insert(0, db)

            if i != 0:
                da_prev = delta @ self.layers[i].T
                delta = da_prev * self.relu_derivative(a_prev)

        return grads_w, grads_b
```

```
    def update_params(self, grads_w, grads_b, lr):
        for i in range(len(self.layers)):
            self.layers[i] -= lr * grads_w[i]
            self.biases[i] -= lr * grads_b[i]

    def train(self, X_train, y_train, X_val, y_val, epochs=10,
learning_rate=0.001):
        history = {"loss": [], "val_loss": [], "accuracy": [],
"val_accuracy": []}
        for epoch in range(epochs):
            y_pred_train = self.forward(X_train)
            loss = self.compute_loss(y_pred_train, y_train)
            acc = self.compute_accuracy(y_pred_train, y_train)

            grads_w, grads_b = self.backward(y_pred_train, y_train)
            self.update_params(grads_w, grads_b, learning_rate)

            y_pred_val = self.forward(X_val)
            val_loss = self.compute_loss(y_pred_val, y_val)
            val_acc = self.compute_accuracy(y_pred_val, y_val)

            history["loss"].append(loss)
            history["val_loss"].append(val_loss)
            history["accuracy"].append(acc)
            history["val_accuracy"].append(val_acc)

            print(f"Epoch {epoch+1:02}/{epochs} "
                  f"| Loss: {loss:.4f} | Acc: {acc:.4f} "
                  f"| Val Loss: {val_loss:.4f} | Val Acc:
{val_acc:.4f}")

        return history

    def predict(self, X):
        y_pred = self.forward(X)
        return np.argmax(y_pred, axis=1)
```

Let's visualize some sample images from our dataset to get a feel for what we're working with.

```
def display_samples(X, y, class_names, img_size=(128, 128),
samples_per_class=2):
    fig, axes = plt.subplots(len(class_names), samples_per_class,
figsize=(12, 12))

    for class_idx, class_name in enumerate(class_names):
        class_indices = np.where(y == class_idx)[0]

        if len(class_indices) >= samples_per_class:
            sample_indices = np.random.choice(class_indices,
```

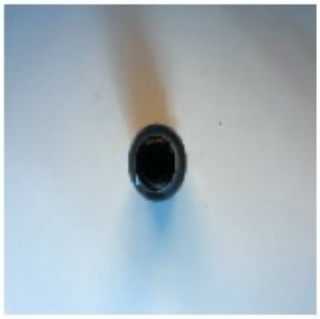
```
samples_per_class, replace=False)

            for i, sample_idx in enumerate(sample_indices):
                img = X[sample_idx].reshape(img_size[0], img_size[1],
3)
                axes[class_idx, i].imshow(img)
                axes[class_idx, i].set_title(class_name)
                axes[class_idx, i].axis('off')

    plt.tight_layout()
    plt.show()

if 'X' in locals() and 'y' in locals() and 'class_names' in locals():
    display_samples(X, y, class_names)
```

cardboard

cardboard

glass

glass



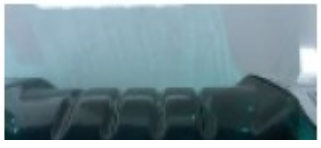
metal

metal

paper

paper

plastic



plastic

Now let's train the MLP on the garbage classification data.

```
if 'X_train' in locals() and 'X_test' in locals():
    # split into training and validation
    X_train, X_val, y_train, y_val = train_test_split(X_train,
y_train, test_size=0.2, random_state=42)

    input_size = X_train.shape[1]
    hidden_sizes = [2056,1028,1028, 512,256,128]
    output_size = len(class_names)

    mlp = MLP(input_size, hidden_sizes, output_size)

    print("Training MLP...")
    history = mlp.train(
        X_train, y_train,
        X_val, y_val,
        epochs=30,
        learning_rate=0.001
    )
    print("Training completed!")
```

```
Training MLP...
Epoch 01/30 | Loss: 1.8961 | Acc: 0.1651 | Val Loss: 1.7966 | Val Acc:
0.2437
Epoch 01/30 | Loss: 1.8961 | Acc: 0.1651 | Val Loss: 1.7966 | Val Acc:
0.2437
Epoch 02/30 | Loss: 1.7658 | Acc: 0.2453 | Val Loss: 1.7525 | Val Acc:
0.2375
Epoch 02/30 | Loss: 1.7658 | Acc: 0.2453 | Val Loss: 1.7525 | Val Acc:
0.2375
Epoch 03/30 | Loss: 1.7002 | Acc: 0.2531 | Val Loss: 1.7218 | Val Acc:
0.2562
Epoch 03/30 | Loss: 1.7002 | Acc: 0.2531 | Val Loss: 1.7218 | Val Acc:
0.2562
Epoch 04/30 | Loss: 1.6730 | Acc: 0.2736 | Val Loss: 1.7550 | Val Acc:
0.2500
Epoch 04/30 | Loss: 1.6730 | Acc: 0.2736 | Val Loss: 1.7550 | Val Acc:
0.2500
Epoch 05/30 | Loss: 1.6722 | Acc: 0.2956 | Val Loss: 1.7578 | Val Acc:
0.2313
Epoch 05/30 | Loss: 1.6722 | Acc: 0.2956 | Val Loss: 1.7578 | Val Acc:
0.2313
Epoch 06/30 | Loss: 1.7183 | Acc: 0.2107 | Val Loss: 1.9552 | Val Acc:
0.2062
Epoch 06/30 | Loss: 1.7183 | Acc: 0.2107 | Val Loss: 1.9552 | Val Acc:
0.2062
Epoch 07/30 | Loss: 1.8443 | Acc: 0.2280 | Val Loss: 1.7720 | Val Acc:
0.2750
Epoch 07/30 | Loss: 1.8443 | Acc: 0.2280 | Val Loss: 1.7720 | Val Acc:
```

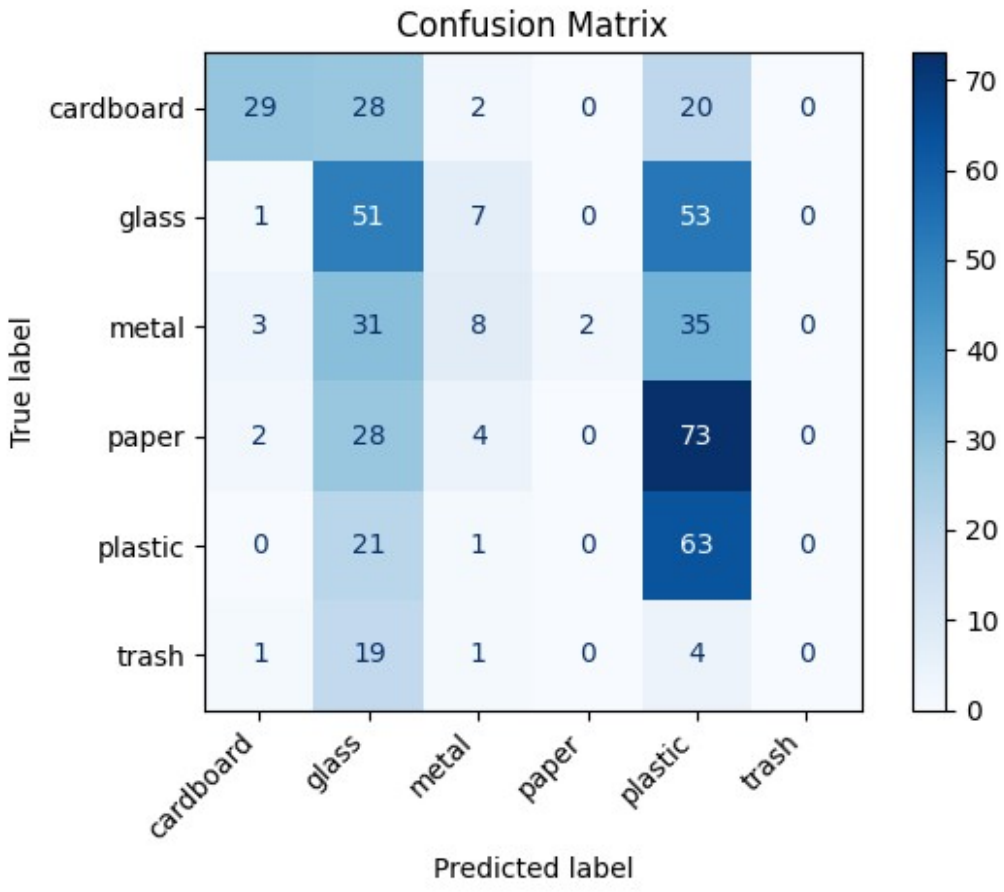
```
0.2750
Epoch 08/30 | Loss: 1.7379 | Acc: 0.3726 | Val Loss: 1.7194 | Val Acc:
0.2562
Epoch 08/30 | Loss: 1.7379 | Acc: 0.3726 | Val Loss: 1.7194 | Val Acc:
0.2562
Epoch 09/30 | Loss: 1.6429 | Acc: 0.2814 | Val Loss: 1.6852 | Val Acc:
0.2625
Epoch 09/30 | Loss: 1.6429 | Acc: 0.2814 | Val Loss: 1.6852 | Val Acc:
0.2625
Epoch 10/30 | Loss: 1.6080 | Acc: 0.3648 | Val Loss: 1.6922 | Val Acc:
0.3125
Epoch 10/30 | Loss: 1.6080 | Acc: 0.3648 | Val Loss: 1.6922 | Val Acc:
0.3125
Epoch 11/30 | Loss: 1.6006 | Acc: 0.3726 | Val Loss: 1.6532 | Val Acc:
0.2875
Epoch 11/30 | Loss: 1.6006 | Acc: 0.3726 | Val Loss: 1.6532 | Val Acc:
0.2875
Epoch 12/30 | Loss: 1.5682 | Acc: 0.3994 | Val Loss: 1.6978 | Val Acc:
0.2875
Epoch 12/30 | Loss: 1.5682 | Acc: 0.3994 | Val Loss: 1.6978 | Val Acc:
0.2875
Epoch 13/30 | Loss: 1.5915 | Acc: 0.3349 | Val Loss: 1.6582 | Val Acc:
0.2500
Epoch 13/30 | Loss: 1.5915 | Acc: 0.3349 | Val Loss: 1.6582 | Val Acc:
0.2500
Epoch 14/30 | Loss: 1.5630 | Acc: 0.3632 | Val Loss: 1.7058 | Val Acc:
0.2375
Epoch 14/30 | Loss: 1.5630 | Acc: 0.3632 | Val Loss: 1.7058 | Val Acc:
0.2375
Epoch 15/30 | Loss: 1.5994 | Acc: 0.3428 | Val Loss: 1.6798 | Val Acc:
0.2750
Epoch 15/30 | Loss: 1.5994 | Acc: 0.3428 | Val Loss: 1.6798 | Val Acc:
0.2750
Epoch 16/30 | Loss: 1.5572 | Acc: 0.3522 | Val Loss: 1.6871 | Val Acc:
0.2812
Epoch 16/30 | Loss: 1.5572 | Acc: 0.3522 | Val Loss: 1.6871 | Val Acc:
0.2812
Epoch 17/30 | Loss: 1.5899 | Acc: 0.3160 | Val Loss: 1.7320 | Val Acc:
0.3063
Epoch 17/30 | Loss: 1.5899 | Acc: 0.3160 | Val Loss: 1.7320 | Val Acc:
0.3063
Epoch 18/30 | Loss: 1.5834 | Acc: 0.4292 | Val Loss: 1.6606 | Val Acc:
0.3187
Epoch 18/30 | Loss: 1.5834 | Acc: 0.4292 | Val Loss: 1.6606 | Val Acc:
0.3187
Epoch 19/30 | Loss: 1.5694 | Acc: 0.3239 | Val Loss: 1.7093 | Val Acc:
0.2938
Epoch 19/30 | Loss: 1.5694 | Acc: 0.3239 | Val Loss: 1.7093 | Val Acc:
0.2938
```

```
Epoch 20/30 | Loss: 1.5555 | Acc: 0.4025 | Val Loss: 1.6216 | Val Acc:
0.3187
Epoch 20/30 | Loss: 1.5555 | Acc: 0.4025 | Val Loss: 1.6216 | Val Acc:
0.3187
Epoch 21/30 | Loss: 1.5139 | Acc: 0.3821 | Val Loss: 1.6462 | Val Acc:
0.3187
Epoch 21/30 | Loss: 1.5139 | Acc: 0.3821 | Val Loss: 1.6462 | Val Acc:
0.3187
Epoch 22/30 | Loss: 1.4922 | Acc: 0.4167 | Val Loss: 1.5981 | Val Acc:
0.3250
Epoch 22/30 | Loss: 1.4922 | Acc: 0.4167 | Val Loss: 1.5981 | Val Acc:
0.3250
Epoch 23/30 | Loss: 1.4703 | Acc: 0.4261 | Val Loss: 1.6449 | Val Acc:
0.3312
Epoch 23/30 | Loss: 1.4703 | Acc: 0.4261 | Val Loss: 1.6449 | Val Acc:
0.3312
Epoch 24/30 | Loss: 1.4811 | Acc: 0.4041 | Val Loss: 1.6292 | Val Acc:
0.2500
Epoch 24/30 | Loss: 1.4811 | Acc: 0.4041 | Val Loss: 1.6292 | Val Acc:
0.2500
Epoch 25/30 | Loss: 1.5001 | Acc: 0.3648 | Val Loss: 1.7402 | Val Acc:
0.3187
Epoch 25/30 | Loss: 1.5001 | Acc: 0.3648 | Val Loss: 1.7402 | Val Acc:
0.3187
Epoch 26/30 | Loss: 1.5993 | Acc: 0.3915 | Val Loss: 1.5920 | Val Acc:
0.3250
Epoch 26/30 | Loss: 1.5993 | Acc: 0.3915 | Val Loss: 1.5920 | Val Acc:
0.3250
Epoch 27/30 | Loss: 1.4367 | Acc: 0.4686 | Val Loss: 1.5905 | Val Acc:
0.3812
Epoch 27/30 | Loss: 1.4367 | Acc: 0.4686 | Val Loss: 1.5905 | Val Acc:
0.3812
Epoch 28/30 | Loss: 1.4285 | Acc: 0.4827 | Val Loss: 1.6415 | Val Acc:
0.3187
Epoch 28/30 | Loss: 1.4285 | Acc: 0.4827 | Val Loss: 1.6415 | Val Acc:
0.3187
Epoch 29/30 | Loss: 1.4742 | Acc: 0.4324 | Val Loss: 1.6448 | Val Acc:
0.3187
Epoch 29/30 | Loss: 1.4742 | Acc: 0.4324 | Val Loss: 1.6448 | Val Acc:
0.3187
Epoch 30/30 | Loss: 1.4900 | Acc: 0.3569 | Val Loss: 1.7397 | Val Acc:
0.2500
Training completed!
Epoch 30/30 | Loss: 1.4900 | Acc: 0.3569 | Val Loss: 1.7397 | Val Acc:
0.2500
Training completed!
```

```python
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

def plot_confusion_matrix(model, X_test, y_test, class_names):
```

```
    y_pred = model.predict(X_test)

    cm = confusion_matrix(y_test, y_pred)

    plt.figure(figsize=(10, 8))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=class_names)
    disp.plot(cmap='Blues', values_format='d')
    plt.title('Confusion Matrix')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
    accuracy = np.sum(np.diag(cm)) / np.sum(cm)
    print(f"Test Accuracy: {accuracy:.4f}")

    class_accuracy = np.diag(cm) / np.sum(cm, axis=1)
    for i, class_name in enumerate(class_names):
        print(f"{class_name} accuracy: {class_accuracy[i]:.4f}")

plot_confusion_matrix(mlp, X_test, y_test, class_names)
```

```
<Figure size 1000x800 with 0 Axes>
```



```
Test Accuracy: 0.3101
cardboard accuracy: 0.3671
glass accuracy: 0.4554
metal accuracy: 0.1013
paper accuracy: 0.0000
plastic accuracy: 0.7412
trash accuracy: 0.0000
```